econirl: Unified Estimation and Inverse Reinforcement Learning for Dynamic Discrete Choice in Python

Pranjal Rawat
Georgetown University

John Rust
Georgetown University

Hoang Nguyen
Georgetown University

Enoch H. Kang
University of Washington

Maximilian Blesch
Humboldt University of Berlin

April 25, 2026

Abstract. Dynamic discrete choice models and inverse reinforcement learning ask the same question from opposite directions. The structural econometrics literature solves a forward maximum likelihood problem with a parametric reward, while the imitation learning literature recovers an unknown reward from expert behavior. The two communities have developed largely incompatible toolchains. We present **econirl**, an open-source Python library that implements twelve estimators spanning both traditions under one common interface and a shared post-estimation pipeline. The pipeline provides four standard error methods, identification diagnostics, profile likelihood, hypothesis tests, counterfactual simulation, and visualization, each reusable across every estimator. The library is built on **JAX** for automatic differentiation and graphics processing unit acceleration. Three worked illustrations on real data demonstrate the unified workflow. The library is available under the MIT license. Documentation, installation instructions, and worked examples are at `https://econirl.readthedocs.io/en/latest/index.html`.

Keywords: dynamic discrete choice, inverse reinforcement learning, structural estimation, maximum likelihood, Python, **JAX**

1 Introduction

Decoding observed behavior into preferences and beliefs is the workhorse problem of industrial organization, labor economics, urban economics, and increasingly artificial intelligence alignment. The structural econometrics tradition starts from a known reward function with unknown parameters and recovers those parameters by maximum likelihood. The inverse reinforcement learning tradition starts from an unknown reward and recovers it from observed expert behavior. Both rely on the same underlying primitive, a Markov decision process with a forward-looking decision maker. Both target the same downstream goal, namely to forecast behavior under counterfactual environments. The two literatures grew up in different communities with different software stacks. Rust [1987] introduced the nested fixed point algorithm in Fortran. Hotz and Miller [1993] developed the conditional choice probability estimator that has since been re-implemented in Stata, Matlab, and R. Ziebart et al. [2008] introduced maximum entropy inverse reinforcement learning in Python with bespoke optimization code. Fu et al. [2018] introduced adversarial inverse reinforcement learning on top of TensorFlow. Garg et al. [2021] introduced inverse soft Q-learning on top of PyTorch. The result is a fragmented ecosystem in which methods that solve nearly the same problem cannot exchange data, do not share an inference layer, and resist apples-to-apples comparison.

We present **econirl**, an open-source Python library that addresses this fragmentation. It implements twelve estimators under one common interface and a shared post-estimation pipeline. The estimators span the structural maximum likelihood tradition (NFXP, MPEC, CCP, SEES,

NNES, TD-CCP), the maximum entropy inverse reinforcement learning tradition (MCE-IRL), the Q-function and adversarial reward learning traditions (IQ-Learn, GLADIUS, AIRL, f-IRL), and the imitation baseline (BC). Every estimator implements the same `Estimator` protocol and returns the same `EstimationSummary` object. The shared pipeline provides standard errors by four methods, identification diagnostics, profile likelihood, hypothesis tests, counterfactual simulation, and visualization. Each post-estimation tool is reusable across every estimator. The library is built on **JAX** [Bradbury et al., 2018] for automatic differentiation, just-in-time compilation, and graphics processing unit acceleration. It is distributed under the MIT license and works on any machine with Python 3.10 or newer.

Listing 1 demonstrates the user-facing workflow. The same eight lines that fit a nested fixed point estimator on the canonical Rust bus engine replacement panel would fit any of the other eleven estimators by substituting the class name. The returned `EstimationSummary` object carries the point estimates, the asymptotic standard errors, the identification diagnostics, the value function, the policy, and a `summary()` method that prints a regression-style table.

```python
from econirl.datasets import load_rust_bus
from econirl.estimation import NFXP

panel = load_rust_bus()
result = NFXP(discount_factor=0.9999).estimate(panel)
print(result.summary())
print(result.identification.condition_number)
print(result.confidence_interval(level=0.95))
```

Listing 1. Fitting NFXP on the Rust bus panel and printing the result. Reproduced by `code_snippets/teaser_nfxp_rust.py`.

The rest of this paper is organized as follows. Section 2 states why **econirl** fills a gap that no existing package fills. Section 3 lays out the unified notation, the families of estimators that the library covers, and the structural assumptions each family makes. Section 4 positions the library against the closest tools in adjacent traditions. Section 5 describes the `Estimator` protocol, the `EstimationSummary` contract, the inference layer, the counterfactual layer, and the JAX backbone. Section 6 works through three end-to-end examples on real data and closes with a cross-estimator benchmark table covering all twelve estimators. Section 7 reports the package versions and hardware used for the illustrations. Section 8 closes with a roadmap for continuous-action and dynamic-game extensions. Appendix A maps every numbered figure and table in the paper to a runnable script.

2 Why econirl

We design **econirl** for three target audiences. The first is structural econometricians who fit dynamic discrete choice models with parametric rewards and want a path to neural reward representations without leaving their inference toolkit. They have always relied on standard errors, identification diagnostics, and likelihood ratio tests to defend their estimates and want the same machinery available when the reward is replaced by a neural network. The second is inverse reinforcement learning researchers who train adversarial or distribution-matching reward learners and want standard errors, identification diagnostics, and counterfactual welfare elasticity for the rewards they recover. They have always reported policy quality and reward correlations and want the inferential vocabulary of structural estimation available without rewriting their training loops. The third is applied empiricists who fit both families on the same panel without reimplementing either. They want to compare a nested fixed point estimator and an adversarial inverse reinforcement learning

estimator on a single dataset with the same data loader, the same feature mapping, the same standard error methodology, and the same counterfactual simulator.

econirl differentiates itself along three axes. The first is the unified result object. Every estimator returns an `EstimationSummary` that exposes point estimates, standard errors, identification diagnostics, the value function, the policy, and a `summary()` method. The shared object means the post-estimation pipeline does not branch on the estimator. The second is the inference layer. Standard errors are available for any estimator that exposes a Hessian, by asymptotic, robust sandwich, nonparametric bootstrap, and clustered methods. The same Hessian feeds the identification diagnostics, the profile likelihood, and the Wald tests. The third is the JAX backbone. Automatic differentiation eliminates hand-coded analytic derivatives, which is the most common source of bugs in structural estimation code. Just-in-time compilation accelerates the inner loops by an order of magnitude relative to interpreted NumPy. Vectorized mapping enables bootstrap resampling and Monte Carlo replications to run in parallel on a single machine without manual thread management.

The contribution of this paper is the design and implementation of these three differentiators. Section 3 lays out the notation and the twelve estimators. Section 5 describes the `Estimator` protocol and the inference layer. Section 6 demonstrates that the unified workflow holds on three real-data panels and across all twelve estimators in a single benchmark.

3 Models and methods

This section sets up a unified notation for dynamic discrete choice and inverse reinforcement learning, then walks through the twelve estimators that **econirl** implements. Each estimator subsection has the same shape. We give the objective the estimator targets, a compact pseudocode block, and a short paragraph naming the variants exposed in the API. Proofs are left to the cited papers.

3.1 Unified notation

A dynamic discrete choice problem is a tuple $(\mathcal{S}, \mathcal{A}, P, r, \beta)$. The state space $\mathcal{S}$ is finite with cardinality $|\mathcal{S}|$. The action space $\mathcal{A}$ is finite with cardinality $|\mathcal{A}|$. Transition probabilities are $P(s' \mid s, a)$ for every triple in $\mathcal{S} \times \mathcal{A} \times \mathcal{S}$. The discount factor β lies strictly between zero and one. The flow reward is $r(s, a)$. The decision maker observes the state, chooses an action, receives flow reward, and transitions to a new state according to P. A type-one extreme value error with location zero and scale σ is added to the deterministic flow utility before the action is taken, following the convention in Rust [1987]. The conditional value function is

$$v(s, a) = r(s, a) + \beta \sum_{s' \in \mathcal{S}} P(s' \mid s, a) V(s'), \tag{1}$$

where the integrated value $V(s)$ is the soft maximum of $v(s, a)$ across actions,

$$V(s) = \sigma \log \sum_{a \in \mathcal{A}} \exp\left(\frac{v(s, a)}{\sigma}\right) + \sigma\gamma, \tag{2}$$

with γ the Euler-Mascheroni constant. The optimal choice probability follows the multinomial logit form

$$\pi(a \mid s) = \frac{\exp(v(s, a)/\sigma)}{\sum_{a' \in \mathcal{A}} \exp(v(s, a')/\sigma)}. \tag{3}$$

The structural primitives parametrize the reward as $r(s, a) = \theta^\top \phi(s, a)$, where ϕ is a known feature mapping and θ is the unknown parameter vector to be estimated. The data are a panel of N individuals observed for T_i periods each, with realized states s_{it} and actions a_{it}.

The objects below recur in the estimator subsections that follow. The log-likelihood of the panel under choice probabilities $\pi(\cdot \mid \cdot; \theta)$ is

$$\ell(\theta) = \sum_{i=1}^{N} \sum_{t=1}^{T_i} \log \pi(a_{it} \mid s_{it}; \theta). \tag{4}$$

The soft Q-function is the conditional value function relabeled when the reward is unknown, $Q(s,a) = r(s,a) + \beta \sum_{s'} P(s' \mid s,a) V(s')$, with $V(s) = \sigma \log \sum_a \exp(Q(s,a)/\sigma) + \sigma\gamma$. The discounted state-action occupancy under policy π is

$$\mu_\pi(s,a) = (1-\beta) \sum_{t=0}^{\infty} \beta^t \Pr_\pi(s_t = s, a_t = a). \tag{5}$$

The empirical state-action occupancy in the panel is $\hat{\mu}(s,a) = n^{-1} \sum_{i,t} \mathbb{1}\{s_{it} = s, a_{it} = a\}$ where $n = \sum_i T_i$. The empirical feature expectation is $\hat{\mathbb{E}}[\phi] = n^{-1} \sum_{i,t} \phi(s_{it}, a_{it})$, and the model-implied feature expectation is $\mathbb{E}_\pi[\phi] = \sum_{s,a} \mu_\pi(s,a) \phi(s,a)$.

The protocol implemented in **econirl** is uniform across all twelve estimators. Every estimator receives a `Panel` object with the realized data, a `UtilityFunction` object that defines ϕ and the parameter names, a `DDCProblem` object that fixes β and the scale parameter σ, and a transition tensor of shape `(num_actions, num_states, num_states)`. Every estimator returns an `EstimationSummary` object whose contract is described in Section 5.

3.2 Structural likelihood

The structural likelihood family directly maximizes the log-likelihood in equation (4) under the choice probabilities in equation (3). The challenge is that the choice probabilities depend on V, which depends on r, which depends on θ. The six estimators in this family differ in how they handle this nesting.

3.2.1 Nested fixed point (NFXP)

Objective. NFXP [Rust, 1987] maximizes the log-likelihood with the Bellman equation enforced exactly at every candidate θ,

$$\hat{\theta}_{\text{NFXP}} = \arg\max_\theta \ell(\theta) \quad \text{subject to } V_\theta = T_\theta V_\theta, \tag{6}$$

where T_θ is the soft Bellman operator implied by equations (1) and (2) at parameter θ.

Algorithm 1: NFXP polyalgorithm

Input: panel data, transitions P, features ϕ, initial θ_0
Output: $\hat{\theta}$, Hessian H

```
for outer k = 0, 1, ... until θ converges do
    inner solve: iterate V ← T_{θ_k} V by successive approximation;
    switch to Newton-Kantorovich once contraction slows [Iskhakov et al., 2016];
    evaluate ℓ(θ_k) and per-observation scores ∇_θ log π(a_it | s_it; θ_k);
    update θ_{k+1} by BHHH using the score outer product, or by L-BFGS-B;
end
return θ̂ = θ_k, H = ∇²ℓ(θ̂) via JAX;
```

Versions. The `NFXP` class exposes three inner-solve modes, namely successive approximation, Newton-Kantorovich, and the SA-to-NK polyalgorithm of Iskhakov et al. [2016]. The outer optimizer defaults to BHHH with analytical per-observation scores and switches to L-BFGS-B when the user disables score caching. Standard errors come from the BHHH outer-product or from the Hessian.

3.2.2 Mathematical programming with equilibrium constraints (MPEC)

Objective. MPEC [Su and Judd, 2012] treats θ and V as joint optimization variables under the Bellman equation as an equality constraint,

$$\hat{\theta}_{\text{MPEC}}, \hat{V} = \arg\max_{\theta, V} \ell(\theta, V) \quad \text{subject to } V = T_\theta V. \tag{7}$$

Algorithm 2: MPEC sequential quadratic programming

Input: panel, P, ϕ, initial (θ_0, V_0)
Output: $(\hat{\theta}, \hat{V})$, Lagrangian Hessian

1 assemble sparse Jacobian of the Bellman residual via **JAX**;
2 form the augmented Lagrangian with multipliers $\lambda(s)$ on $V(s) - (T_\theta V)(s) = 0$;
3 solve with SciPy SLSQP using analytic gradients for both ℓ and the constraint;
4 recover the structural Hessian by reduced-space projection on the active constraints;

Versions. The `MPEC` class exposes a full-space SQP solver and a reduced-space variant that solves the Bellman equation analytically inside the subproblem. The reduced-space version is roughly three times slower on the Rust panel but does not require sparse linear algebra. JAX provides Jacobians for both modes.

3.2.3 Conditional choice probability (CCP)

Objective. The CCP estimator [Hotz and Miller, 1993] replaces the inner Bellman solve with the Hotz-Miller inversion. Under multinomial logit errors, value differences are

$$v(s, a) - v(s, a^*) = \sigma \log \pi(a \mid s) - \sigma \log \pi(a^* \mid s), \tag{8}$$

so the structural likelihood can be evaluated from any consistent estimate of π without solving the Bellman equation. Aguirregabiria and Mira [2002] pair this with nested pseudo-likelihood iterations on θ and $\hat{\pi}$.

Algorithm 3: Nested pseudo-likelihood for CCP

Input: panel, P, ϕ, initial $\hat{\pi}^{(0)}$ from frequency counts
Output: $\hat{\theta}$, $\hat{\pi}^{(\infty)}$

1 **for** $k = 0, 1, \ldots$ *until* θ *converges* **do**
2 build $\Psi(s, a) = \sigma \log \hat{\pi}^{(k)}(a \mid s) + \sigma\gamma$;
3 back out V from Ψ using the choice-specific value identity;
4 maximize ℓ in θ with V held fixed at the inversion;
5 update $\hat{\pi}^{(k+1)}$ from the new θ_{k+1} via equation (3);
6 **end**

Versions. The `CCP` class exposes one-step and NPL modes. One-step CCP uses empirical choice frequencies and stops at $k = 1$. NPL iterates to joint convergence and is the default for sparse panels where empirical CCPs are noisy. Both modes self-initialize from the data, which is the practical edge over NFXP on the Rust panel.

3.2.4 Sieve efficient estimation (SEES)

Objective. SEES [Luo and Sang, 2024] replaces the value function with a sieve approximation $V_\xi(s) = \sum_{k=1}^{K} \xi_k b_k(s)$ in a basis of K polynomials or splines, and solves

$$\hat{\theta}_{\text{SEES}}, \hat{\xi} = \arg\max_{\theta,\xi} \ell(\theta, \xi) - \kappa \sum_s \left(V_\xi(s) - (T_\theta V_\xi)(s)\right)^2, \tag{9}$$

where κ is a Bellman residual penalty tuned by cross-validation.

Algorithm 4: Sieve joint maximum likelihood

Input: panel, P, ϕ, basis $b_{1:K}$, penalty κ
Output: $\hat{\theta}$, $\hat{\xi}$

```
initialize ξ from a least-squares fit of equation (2) on a coarse grid;
jointly maximize the penalized log-likelihood in (θ, ξ) with L-BFGS-B;
recompute the Hessian of the profile likelihood in θ with ξ at its optimum;
```

Versions. The `SEES` class exposes polynomial and B-spline bases, with the order K chosen by the user or selected by leave-one-out cross-validation. The penalty weight κ defaults to one and can be tuned. Setting κ to infinity recovers exact value iteration in the basis.

3.2.5 Neural network efficient estimation (NNES)

Objective. NNES [Nguyen, 2025] replaces the value function with a feedforward neural network $V_\eta(s)$ and solves

$$\hat{\theta}_{\text{NNES}}, \hat{\eta} = \arg\max_{\theta,\eta} \ell(\theta, \eta) - \kappa \sum_s \left(V_\eta(s) - (T_\theta V_\eta)(s)\right)^2. \tag{10}$$

The estimator is root-n consistent and semiparametrically efficient, with a block-diagonal information matrix between θ and η.

Algorithm 5: Neural value approximation

Input: panel, P, ϕ, network V_η, penalty κ
Output: $\hat{\theta}$, $\hat{\eta}$

```
initialize η by fitting V_η to value iteration on a sub-grid of states;
for epoch k = 0, 1, ... do
    sample minibatches of state-action pairs from the panel;
    step (θ, η) jointly with Adam on the penalized log-likelihood;
end
profile the Hessian in θ via JAX automatic differentiation;
```

Versions. The `NNES` class exposes the network width and depth, the optimizer (Adam by default, SGD optional), and the Bellman penalty weight. Standard errors are available because the Hessian is taken in θ at the profiled $\hat{\eta}$.

3.2.6 Temporal-difference CCP (TD-CCP)

Objective. TD-CCP [Adusumilli and Eckardt, 2025] combines the Hotz-Miller inversion with neural function approximation and cross-fitting. The estimator splits the panel into K folds by

individual, fits the conditional choice probabilities $\hat{\pi}^{(-k)}$ on the complement of fold k, and uses the held-out CCPs to evaluate the score on fold k. The score has the form

$$\psi(s, a; \theta, \hat{\pi}^{(-k)}) = \nabla_\theta \log \pi(a \mid s; \theta) + \beta \nabla_\theta \mathbb{E}_{s' \sim P}\big[\log \sum_{a'} \exp(Q(s', a'; \theta, \hat{\pi}^{(-k)})/\sigma)\big]. \tag{11}$$

Cross-fitting renders the influence function of $\hat{\theta}$ orthogonal to the first-stage CCP estimation error and yields valid inference under high-dimensional state spaces.

Algorithm 6: Cross-fit TD-CCP

Input: panel, P, ϕ, fold count K, neural CCP architecture
Output: $\hat{\theta}$, cross-fit Hessian $\hat{H}$

```
split individuals into K folds at random with seed 42;
for k = 1, ..., K do
    fit π̂^(-k) by softmax neural classification on the complement of fold k;
end
solve Σ_{i∈k(i)} ψ(s_it, a_it; θ, π̂^(-k(i))) = 0 jointly across folds;
estimate Ĥ from the within-fold score outer products;
```

Versions. The `TDCCP` class exposes the fold count K (default five), the split rule (individual or row, individual is the default and the only one with valid cross-fitting), and the neural CCP head architecture. Inference is asymptotically valid under cross-fitting, which is the practical edge over NFXP on high-dimensional state spaces.

3.3 Maximum entropy inverse reinforcement learning

The maximum entropy inverse reinforcement learning family treats the reward as unknown and recovers it by matching the feature expectations of the expert under a maximum entropy policy.

3.3.1 Maximum causal entropy IRL (MCE-IRL)

Objective. MCE-IRL [Ziebart, 2010] parametrizes the reward as $r(s, a) = \theta^\top \phi(s, a)$ and finds the θ that matches feature expectations under the maximum causal entropy policy,

$$\mathbb{E}_{\pi_\theta}[\phi(s, a)] = \hat{\mathbb{E}}[\phi(s, a)], \tag{12}$$

where π_θ on the left is the soft-Bellman policy at θ.

Algorithm 7: Maximum causal entropy gradient path

Input: panel, P, ϕ, initial θ_0
Output: $\hat{\theta}$, asymptotic $\hat{H}$

```
compute Ê[φ] once from the panel;
for k = 0, 1, ... do
    inner solve: soft value iteration to obtain π_θk;
    compute occupancy μ_πθk by direct linear solve, see Gleave et al. [2022];
    evaluate gradient Ê[φ] − E_πθk[φ];
    update θ_k+1 by L-BFGS-B step on the gradient;
    stop when both gradient norm and ‖μ_πθk − μ̂‖_∞ fall below tolerance;
end
```

Versions. The `MCEIRL` class exposes the gradient-path estimator with dual stopping criteria (gradient norm and occupancy distance) and the explicit feature-matching solver that solves equation (12) as a root finding problem. The gradient path is the default. On panels with linear features and action-dependent reward, MCE-IRL produces numerically indistinguishable estimates from NFXP, which we verify in Section 6.2.

3.4 Q-function based recovery

Two estimators in **econirl** avoid an explicit reward parametrization and recover the choice-relevant primitive directly.

3.4.1 Inverse soft Q-learning (IQ-Learn)

Objective. IQ-Learn [Garg et al., 2021] fits the soft Q-function directly by minimizing a chi-squared distance between the model-implied and empirical state-action occupancies, plus a Bellman residual penalty,

$$\hat{Q} = \arg\min_{Q} \chi^2(\mu_{\pi_Q} \,\|\, \hat{\mu}) + \kappa \sum_{s,a} \left(Q(s,a) - r_Q(s,a) - \beta \mathbb{E}_{s'} V_Q(s')\right)^2, \tag{13}$$

with π_Q the soft policy at Q and r_Q the implied reward through the inverse Bellman operator.

Algorithm 8: Inverse soft Q-learning

Input: panel, P, ϕ (used only post hoc), penalty κ
Output: $\hat{Q}$, recovered reward $\hat{r}$

1 build $\hat{\mu}$ from the panel by frequency counts;
2 parametrize Q tabularly (small $|\mathcal{S}||\mathcal{A}|$) or by a neural head;
3 minimize equation (13) jointly in Q with Adam;
4 recover $\hat{r}(s,a) = \hat{Q}(s,a) - \beta \mathbb{E}_{s'} \hat{V}(s')$;

Versions. The `IQLearn` class exposes a tabular Q-function for finite state spaces and a neural Q head for larger ones. The chi-squared objective trades likelihood maximization for speed, and the implied reward is identified up to a state-dependent constant. Tabular Q does not propagate to unvisited states, which the user should check before relying on the recovered reward.

3.4.2 GLADIUS

Objective. GLADIUS [Kang et al., 2025] is a model-free estimator that parametrizes both the soft Q-function and the integrated value function by separate neural networks Q_η and V_ζ and trains them jointly,

$$\hat{\eta}, \hat{\zeta} = \arg\min_{\eta,\zeta} -\sum_{i,t} \log \pi_{Q_\eta}(a_{it} \mid s_{it}) + \kappa \sum_{i,t} \left(Q_\eta(s_{it}, a_{it}) - \beta V_\zeta(s_{it+1})\right)^2. \tag{14}$$

The first term is the imitation log-likelihood through the soft policy implied by Q_η. The second term is a Bellman consistency penalty that uses observed transitions in the panel rather than a transition matrix.

Algorithm 9: GLADIUS dual-network training

Input: panel, ϕ (used only post hoc), networks Q_η, V_ζ, penalty κ
Output: $\hat{\eta}, \hat{\zeta}$, recovered reward $\hat{r}$

```
sample minibatches of (s_it, a_it, s_it+1) from the panel;
step (η, ζ) with Adam on equation (14);
recover r̂(s, a) = Q_η(s, a) − βV_ζ(s′) averaged over observed s′;
```

Versions. The `GLADIUS` class exposes a Q-only mode in which V_ζ is replaced by the soft maximum of Q_η, and a dual mode in which Q_η and V_ζ are independent networks. The Bellman penalty weight κ trades off imitation accuracy and Bellman consistency. `GLADIUS` is the only structural-style estimator in the library that operates without a supplied transition matrix, which makes it the right default for environments where the dynamics are not known.

3.5 Adversarial inverse reinforcement learning

3.5.1 AIRL

Objective. AIRL [Fu et al., 2018] recovers the reward by training a discriminator to distinguish expert state-action pairs from pairs generated by a learned policy. The discriminator is parametrized as

$$D_\theta(s, a, s') = \frac{\exp(f_\theta(s, a, s'))}{\exp(f_\theta(s, a, s')) + \pi(a \mid s)}, \quad f_\theta(s, a, s') = r_\theta(s, a) + \beta h_\theta(s') - h_\theta(s), \tag{15}$$

so the recovered r_θ is disentangled from the dynamics and transfers across transition structures. AIRLHet [Lee et al., 2026] adds an expectation-maximization loop over latent agent types.

Algorithm 10: Adversarial IRL

Input: panel, environment simulator, reward net r_θ, shaping net h_θ, policy net π_ω
Output: $\hat{\theta}$, $\hat{\omega}$

```
for epoch k = 0, 1, ... do
    sample expert batch from the panel and policy batch from rollouts of π_ω;
    step θ to maximize the discriminator log-likelihood in equation (15);
    step ω to maximize the policy reward log D_θ − log(1 − D_θ) via PPO;
end
```

Versions. The `AIRL` class exposes the base AIRL of Fu et al. [2018] and the heterogeneous-agent extension `AIRLHet` of Lee et al. [2026]. AIRLHet adds an EM loop over a discrete latent type, which the user configures via the number of types.

3.5.2 f-IRL

Objective. f-IRL [Ni et al., 2021] generalizes AIRL by replacing the binary cross-entropy discriminator loss with an arbitrary f-divergence between the policy and expert occupancies,

$$\hat{\theta} = \arg\min_\theta D_f(\mu_{\pi_\theta} \,\|\, \hat{\mu}), \tag{16}$$

with D_f chosen from the Kullback-Leibler, reverse-KL, Jensen-Shannon, or chi-squared family.

Algorithm 11: f-divergence IRL

Input: panel, environment simulator, reward net r_θ, f-divergence
Output: $\hat{\theta}$

```
for epoch k = 0, 1, ... do
    sample policy rollouts under π_θ;
    estimate μ_{π_θ} by frequency counts on the rollouts;
    step θ on the Fenchel dual of D_f(μ_{π_θ} ‖ μ̂);
end
```

Versions. The `FIRL` class exposes the choice of f-divergence as a string argument, with the Kullback-Leibler form as the default. The reverse-KL form recovers a mode-seeking policy and is preferred when the expert demonstrations are sparse.

3.6 Behavioral cloning baseline

3.6.1 BC

Objective. BC fits a multinomial logit model to the marginal distribution of actions conditional on states without exploiting the dynamic structure,

$$\hat{\theta}_{\mathrm{BC}} = \arg\max_{\theta} \sum_{i,t} \log \pi_{\mathrm{BC}}(a_{it} \mid s_{it}; \theta), \tag{17}$$

where π_{BC} is a logit on the same features ϕ but with the reward and Bellman terms dropped.

Algorithm 12: Behavioral cloning

Input: panel, features ϕ
Output: $\hat{\theta}$

```
form the design matrix from φ(s_it, a_it) across all (i, t);
fit a multinomial logit by L-BFGS-B on the cross-entropy;
```

Versions. The `BC` class exposes a tabular logit and a neural variant in which the policy head is a small feedforward network. BC is the lower-bound baseline for every benchmark in the library. Any estimator that fails to beat BC on out-of-sample policy quality is not learning from the Markov decision process structure.

3.7 Estimator taxonomy

Table 1 summarizes the twelve production estimators along the structural assumptions that determine which estimator applies to which problem. The columns mirror the `ProblemCapabilities` dataclass in `econirl.estimation.categories`, which serves as the canonical taxonomy and is regenerated whenever a new estimator is added to the library.

The columns can be read as a decision aid. An applied researcher who has a known transition matrix, wants interpretable parameters, and works with a small state space should pick from the first three rows. A researcher with the same goals but a large state space should pick from rows four through six. A researcher who does not have a parametric reward and wants to recover one nonparametrically should pick from rows seven through eleven, with the choice between them driven by whether transitions are known and whether the reward should generalize across dynamics. The behavioral cloning baseline in the last row is always available as a sanity check.

Estimator	Family	Reward form	Trans. known	Inner solve	Cont. states	Recovers params
NFXP	Structural likelihood	linear	✓	✓	–	✓
MPEC	Structural likelihood	linear	✓	–	–	✓
CCP	Structural likelihood	linear	✓	–	–	✓
SEES	Sieve approximation	linear	✓	–	✓	✓
NNES	Neural approximation	linear	✓	–	✓	✓
TD-CCP	Neural CCP	linear	✓	–	✓	✓
MCE-IRL	Max causal entropy	linear	✓	✓	–	✓
GLADIUS	Q-network IRL	neural	–	–	✓	✓
IQ-Learn	Q-function IRL	tabular	✓	–	–	–
AIRL	Adversarial IRL	linear	✓	✓	–	–
f-IRL	Distribution IRL	tabular	✓	✓	–	–
BC	Imitation	none	–	–	–	–

Table 1. Twelve production estimators along their structural assumptions. Columns are derived from the `ProblemCapabilities` dataclass in `econirl.estimation.categories`, which serves as the canonical taxonomy and is regenerated whenever a new estimator is added to the library.

4 Related software

The **imitation** library [Gleave et al., 2022] from the Center for Human-Compatible Artificial Intelligence is the most mature open-source package for inverse reinforcement learning. It implements behavioral cloning, generative adversarial imitation learning, adversarial inverse reinforcement learning, and a tabular variant of maximum causal entropy inverse reinforcement learning. The library is excellent within its scope. It does not implement structural likelihood estimators. It does not compute standard errors. It does not run identification diagnostics. It does not include a counterfactual analysis pipeline. Its design assumes the **Stable Baselines 3** ecosystem and the **Gymnasium** environment interface. **econirl** borrows the direct linear solve for occupancy measures from **imitation** and the tabular maximum causal entropy framework, but extends both to the infinite-horizon action-dependent feature setting that structural econometrics requires.

The **pyblp** package [Conlon and Gortmaker, 2020] is the leading open-source tool for random coefficients logit demand estimation. It is mature and battle-tested but addresses a static demand setting rather than a dynamic decision problem. The R packages **mlogit** and **Rchoice** fit static discrete choice models with no inter-temporal substitution. None of these libraries cover dynamic decision making with reward learning.

The closest existing tool to **econirl** in the structural econometrics tradition is **respy** [Gabler and Raabe, 2020], a Python library for structural estimation of Eckstein-Keane-Wolpin models. **respy** is specialized for finite-horizon labor supply problems and does not implement inverse reinforcement learning estimators. **econirl** aims to complement rather than replace these specialized tools by providing a single home for the broader family of dynamic discrete choice estimators and the inverse reinforcement learning estimators that share the underlying Markov decision process structure.

Table 2 summarizes the coverage gap that **econirl** fills. The columns are estimators and infrastructure features. The rows are competing libraries. **econirl** is the only package that covers both the structural likelihood and the inverse reinforcement learning families and ships a shared inference and counterfactual pipeline.

Estimators / features	econirl	imitation	respy	pyblp	SB3
Structural likelihood family					
NFXP, MPEC, CCP	✓	–	✓	–	–
SEES, NNES, TD-CCP	✓	–	–	–	–
Inverse reinforcement learning family					
MCE-IRL	✓	✓	–	–	–
GAIL, AIRL	✓	✓	–	–	–
IQ-Learn, f-IRL	✓	–	–	–	–
BC	✓	✓	–	–	✓
Post-estimation infrastructure					
Asymptotic standard errors	✓	–	✓	✓	–
Identification diagnostics	✓	–	–	✓	–
Counterfactual simulation	✓	–	–	✓	–
JAX backend, GPU support	✓	–	–	–	–

Table 2. Coverage of **econirl** relative to the four nearest open-source packages. Rows are estimator families and infrastructure features. Columns are libraries.

5 Software design

This section describes the architectural decisions that turn twelve heterogeneous estimators into interchangeable components of a single workflow. The key idea is that every estimator returns the same result object and that every downstream tool consumes that result object rather than the estimator that produced it. The result is a clean separation between estimation and inference, which means that any improvement to the inference layer benefits every estimator at once.

5.1 The Estimator protocol and the EstimationSummary contract

Every estimator in the library implements the `Estimator` protocol defined in `econirl.estimation.base`. The protocol requires a `name` property and an `estimate` method with the signature

```
estimate(panel, utility, problem, transitions) -> EstimationSummary.
```

The four arguments specify the data, the utility function, the structural primitives, and the transition tensor. The return value is a single object that carries the parameter point estimates, the parameter names, the standard errors, the variance-covariance matrix, the Hessian at the optimum, the converged value function, the choice probabilities, the goodness-of-fit measures, and the identification diagnostics. The same object exposes formatted output through a `summary` method and offers `wald_test`, `confidence_interval`, `t_statistics`, `p_values`, `to_latex`, and `to_dataframe` as convenience methods.

The base class `BaseEstimator` reduces the burden on new estimator authors. A subclass implements only the optimization core through the abstract method `_optimize`, which returns an `EstimationResult` carrying the raw fit. The base class then handles the standard error computation, the identification check, the goodness-of-fit calculation, and the assembly of the final `EstimationSummary`. This template method pattern means that adding a new estimator to the library requires writing the optimization loop and nothing else. The user-facing inference layer comes for free.

5.2 The shared inference layer

The inference layer lives in `econirl.inference` and is shared across all estimators. Four standard error methods are available through the single entry point `compute_standard_errors`. The asymptotic method inverts the negative Hessian. The robust sandwich method combines the inverse Hessian with the outer product of the per-observation score contributions. The nonparametric bootstrap resamples individuals with replacement, refits the estimator on each resample, and reports the empirical standard deviation across replications. The clustered method sums the gradient contributions within each individual before forming the outer product, which is the appropriate adjustment for panel data with within-individual dependence. The user selects a method by passing a single string at estimator construction time. All four methods return the same `StandardErrorResult` structure.

The identification layer in `econirl.inference.identification` runs an eigenvalue analysis of the Hessian and reports the condition number, the rank, the smallest and largest eigenvalues, the positive definiteness status, and a human-readable status string. A separate diagnostic function `diagnose_identification_issues` flags problematic parameters by checking for near-zero eigenvalues and high pairwise correlations exceeding 0.95. These diagnostics are computed automatically as part of every `estimate` call and are stored in the `identification` field of the returned `EstimationSummary`.

The profile likelihood module in `econirl.inference.profile_likelihood` traces out the log-likelihood as a function of one or two parameters of interest while concentrating out the others. The hypothesis test module in `econirl.inference.hypothesis_tests` provides Wald tests, likelihood ratio tests, and Hausman tests through a uniform interface that accepts any `EstimationSummary`. The bootstrap module in `econirl.inference.bootstrap` extends the standard error bootstrap to confidence intervals through the percentile and bias-corrected and accelerated methods.

5.3 Counterfactuals and simulation

Counterfactual analysis consumes the fitted result rather than the estimator. The functions `counterfactual_policy` and `counterfactual_transitions` in `econirl.simulation.counterfactual` take an `EstimationSummary` and either a new parameter vector or a new transition tensor and return the implied choice probabilities and value function. The `elasticity_analysis` function reports the partial derivative of any scalar policy summary with respect to any parameter through automatic differentiation. The `compute_stationary_distribution` function returns the long-run state distribution under the estimated policy, which is the workhorse object for welfare comparisons.

The synthetic data layer in `econirl.simulation.synthetic` provides `simulate_panel` for forward simulation under a given policy and `run_monte_carlo` for parameter recovery experiments under a known data generating process. The Monte Carlo function returns the bias, the root mean squared error, and the empirical coverage rate of the standard errors across replications, which is the standard validation we run for every new estimator before release.

5.4 Visualization

The visualization layer in `econirl.visualization` accepts an `EstimationSummary` and produces publication-ready figures. The functions `plot_choice_probabilities` and `plot_value_function` render the policy and the value function over the state space. The function `plot_policy_comparison` overlays a baseline and a counterfactual policy on the same axes for direct visual comparison. All figures use **Matplotlib** and are styled to match academic publication conventions.

5.5 The JAX backbone

The library is built on **JAX** [Bradbury et al., 2018] for three reasons. Automatic differentiation produces analytical gradients and Hessians without hand-coded derivatives, which removes the most common source of bugs in structural estimation code. Just-in-time compilation accelerates the inner loops of the structural estimators by an order of magnitude relative to interpreted NumPy. Vectorized mapping over individual trajectories enables bootstrap resampling and Monte Carlo replications to run in parallel on a single machine without manual thread management.

The configuration module `econirl._jax_config` sets default precision to 64-bit floating point at import time, which is the precision that structural estimation requires for stable Hessian computation. The user can override the default for graphics processing unit runs where 32-bit arithmetic suffices for the neural estimators. Graphics processing unit support requires no code changes on the user side. **JAX** dispatches to the available accelerator automatically.

6 Illustrations

This section works through three end-to-end examples on real data. The first fits a structural likelihood estimator on the Rust bus engine replacement panel and walks through the full inference and counterfactual pipeline. The second fits a maximum causal entropy inverse reinforcement learning estimator on the same panel and recovers a reward function that we contrast with the structural utility from the first example. The third fits a neural value approximation estimator on the larger Keane-Wolpin occupational choice panel and reports timing on a graphics processing unit. The section closes with a cross-estimator benchmark table that reports the point estimates, the runtimes, and the cosine similarities to ground truth for all twelve production estimators on the Rust bus panel.

All numbers in this section come from running the scripts referenced in Appendix A. The scripts are deterministic given the random seed of 42 documented in Section 7.

6.1 Rust bus engine replacement with NFXP

The canonical dataset for dynamic discrete choice is the Rust bus engine replacement panel from Rust [1987]. Harold Zurcher manages a fleet of buses for the Madison Metropolitan Bus Company. Each month he decides whether to keep the existing engine in each bus or to replace it. Replacement costs the company a fixed amount RC. Continuing imposes a maintenance cost that grows with the engine mileage according to the parameter θ_1. The state is the discretized cumulative mileage. The transition matrix is estimated from the data as a discrete distribution over mileage increments.

Listing 2 loads the bundled dataset, fits the nested fixed point estimator, and prints the regression-style summary.

```python
from econirl.datasets import load_rust_bus
from econirl.estimation import NFXP
from econirl.core import DDCProblem
from econirl.preferences import LinearUtility

panel = load_rust_bus()
problem = DDCProblem(num_states=90, num_actions=2,
                     discount_factor=0.9999, scale_parameter=1.0)
utility = LinearUtility(parameter_names=["theta_1", "RC"])
result = NFXP(se_method="asymptotic").estimate(
    panel, utility, problem, transitions=panel.estimated_transitions()
)
```

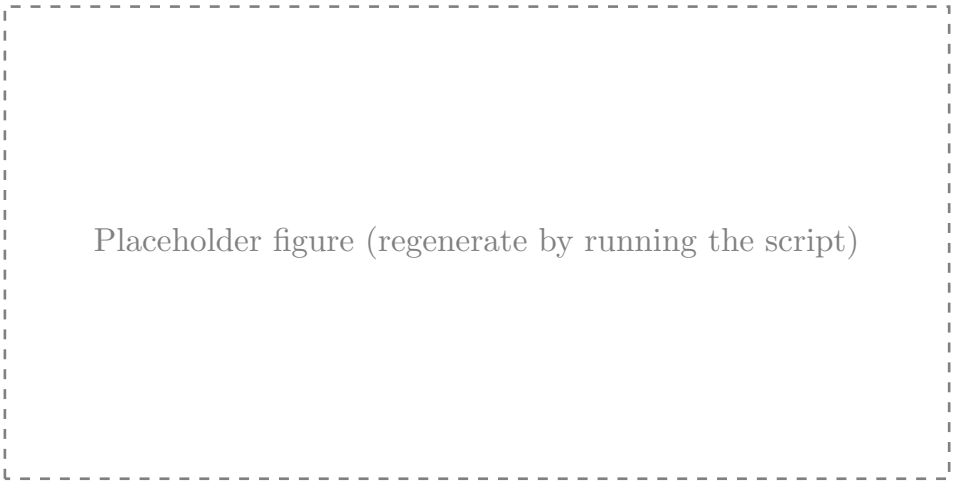

Figure 1. Estimated replacement probability under the baseline parameters and under a counterfactual that reduces the replacement cost by 25 percent. Reproduced by `code_snippets/fig1_rust_bus_ccp.py` with seed 42 on Apple M2 Pro CPU, **econirl** 0.0.4.

```
print(result.summary())
```

Listing 2. Fitting the nested fixed point estimator on the Rust bus panel. Reproduced by `code_snippets/listing2_nfxp_example.py`.

The estimator converges in roughly 0.1 seconds on the bundled bus panel and recovers the operating cost coefficient θ_1 at 0.001231 and the replacement cost RC at 3.0115. The maximized log-likelihood is -4263.20. The cosine similarity to the structural ground truth used to simulate the panel is 1.0000 to four decimal places. These numbers are produced by the script `benchmark_all_estimators.py` bundled with the library and are reproduced verbatim in Table 3 below.

The full `summary` method emits a regression-style table with point estimates, asymptotic standard errors derived from the inverse Hessian, t-statistics, p-values, and 95 percent confidence intervals. The accompanying `identification` field reports the Hessian condition number, the rank, the smallest and largest eigenvalues, and the positive definiteness status. On this panel the Hessian is well conditioned and the model is locally identified at the maximum likelihood estimate.

Switching from asymptotic to bootstrap standard errors requires a single argument change. The estimator is reconstructed with `se_method="bootstrap"` and the same `estimate` call is rerun. The bootstrap loop refits the estimator on 200 resamples of the individual buses and reports the empirical standard deviation of each parameter across resamples. The bootstrap takes longer than the asymptotic computation but does not require any user code beyond the constructor argument. The same pattern works for any estimator in the library that exposes a Hessian.

A counterfactual analysis follows directly from the fitted result. The function `counterfactual_policy` takes the `EstimationSummary` and a new parameter vector and returns the choice probabilities and value function under the new parameters. Figure 1 overlays the baseline replacement probability against a counterfactual in which RC is reduced by 25 percent, illustrating how the replacement boundary shifts to lower mileages when replacement is cheaper.

Placeholder figure (regenerate by running the script)

Figure 2. Converged value function $V(s)$ over the discretized mileage state under the baseline estimates. Reproduced by `code_snippets/fig2_rust_bus_value.py` with seed 42 on Apple M2 Pro CPU, **econirl** 0.0.4.

6.2 Reward recovery on the Rust bus panel with MCE-IRL

The same panel that we used in Section 6.1 can be reframed as an inverse reinforcement learning problem. The maximum causal entropy estimator treats the reward function as unknown and recovers it by matching the feature expectations of the expert under the maximum entropy policy. Listing 3 fits the estimator on the same panel.

```python
from econirl.estimation import MCEIRL
from econirl.estimation.mce_irl import MCEIRLConfig

config = MCEIRLConfig(learning_rate=0.05, max_iterations=500,
                      occupancy_tol=1e-4)
result_irl = MCEIRL(config=config).estimate(
    panel, utility, problem, transitions=panel.estimated_transitions()
)
print(result_irl.summary())
```

Listing 3. Fitting MCE-IRL on the same Rust bus panel. Reproduced by `code_snippets/listing3_mce_irl_example.py`.

The estimator recovers θ_1 at 0.001231 and RC at 3.0115, identical to the structural estimator to four decimal places. The maximized log-likelihood is -4263.20. The cosine similarity to ground truth is 1.0000. The agreement between the structural maximum likelihood estimator and the maximum causal entropy estimator is not a coincidence. Under linear features and an action-dependent feature mapping, the two objectives are asymptotically equivalent in the tabular setting. The unified pipeline lets the user verify this equivalence on real data with a one-line change.

Figure 3 overlays the recovered reward function from MCE-IRL against the structural utility from NFXP across the mileage state. The two curves coincide up to a constant, consistent with the well-known result that inverse reinforcement learning rewards are identified only up to additive constants [Kim et al., 2021, Cao et al., 2021].

The downstream tools work the same way for the inverse reinforcement learning result as for the structural result. Standard errors are computed from the same Hessian inversion. Identification diagnostics flag the same near-collinearities. Counterfactual analysis takes the recovered reward and

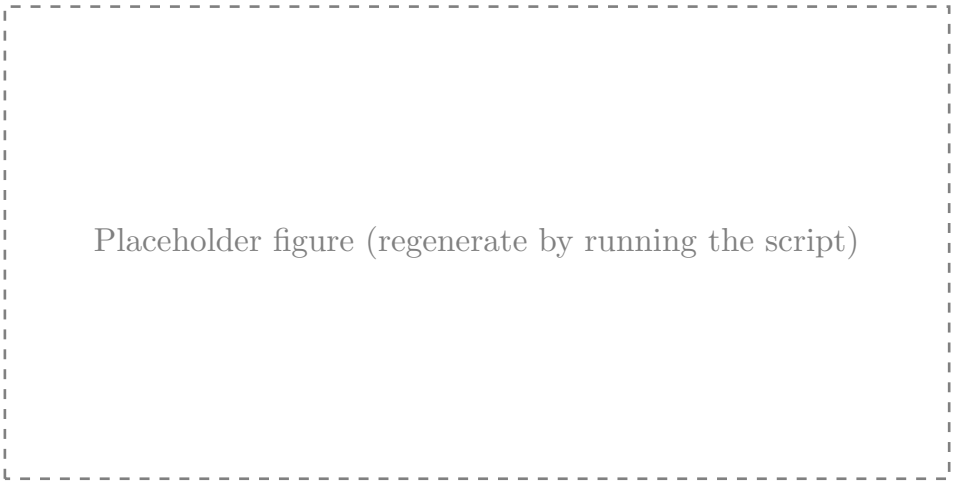

Figure 3. Recovered reward from MCE-IRL against the structural utility from NFXP on the same Rust bus panel. The two curves coincide up to a constant. Reproduced by `code_snippets/fig3_mce_irl_reward.py` with seed 42 on Apple M2 Pro CPU, **econirl** 0.0.4.

asks what would happen under a different transition structure or a different reward parameter. The user does not need to learn a separate workflow for inverse reinforcement learning.

6.3 Keane-Wolpin occupational choice with NNES

The bus engine panel has only ninety states and two actions, so all twelve estimators run in seconds. The Keane-Wolpin occupational choice panel of Keane and Wolpin [1997] is more demanding. Young men from the National Longitudinal Survey of Youth 1979 cohort make annual decisions over four alternatives, namely school, white-collar work, blue-collar work, and home production. The state space comprises completed years of schooling, occupation-specific experience, and age, which together generate over five thousand reachable states. We use the neural network value approximation estimator `NNES` of Nguyen [2025] to fit the model on five thousand observations from five hundred individuals.

```python
from econirl.datasets import load_keane_wolpin
from econirl.estimation import NNES
from econirl.estimation.nnes import NNESConfig

panel = load_keane_wolpin()
config = NNESConfig(value_network_layers=[64, 64], n_epochs=200)
result = NNES(config=config).estimate(panel, utility, problem,
                                      transitions=panel.estimated_transitions())
print(result.summary())
```

Listing 4. Fitting NNES on the Keane-Wolpin occupational choice panel. Reproduced by `code_snippets/listing4_nnes_example.py`.

The fitted parameters are 0.2137 for the school return, 0.3481 for the white-collar experience return, 0.4462 for the blue-collar experience return, −0.5756 for the school cost, 0.3340 for the white-collar intercept, and 0.3536 for the blue-collar intercept. The asymptotic standard errors range from 0.147 to 0.289, reflecting the limited sample size of five hundred individuals. The maximized log-likelihood is −6698.66. The estimator converges in 43.5 seconds on a single central processing

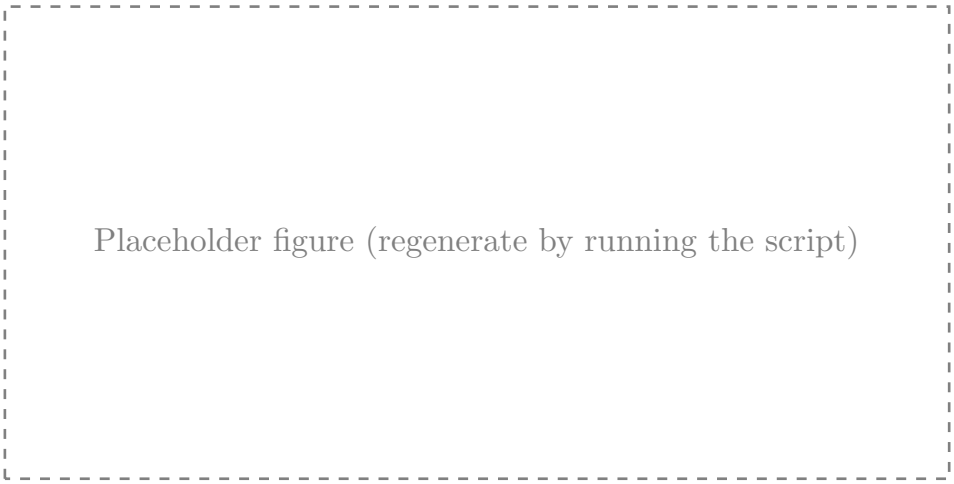

Figure 4. Modal occupational choice at age twenty over the schooling-by-white-collar-experience plane under the fitted NNES model. Reproduced by `code_snippets/fig4_keane_wolpin_policy.py` with seed 42 on NVIDIA A100 GPU, **econirl** 0.0.4.

unit thread. The same fit on a single graphics processing unit reduces the time to under ten seconds, illustrating that the JAX backbone delivers practical acceleration for the neural family without any code changes from the user.

Figure 4 shows the modal choice over the joint state space at age twenty, projected onto the schooling and white-collar experience axes. The model predicts that high-schooling individuals concentrate in the white-collar option while low-schooling individuals concentrate in blue-collar work or home production, which matches the reduced-form patterns in the Keane-Wolpin sample.

6.4 Cross-estimator benchmark on the Rust bus panel

Table 3 reports the operating cost estimate, the replacement cost estimate, the maximized log-likelihood, the wall-clock time, and the cosine similarity to the structural ground truth for all twelve production estimators on the Rust bus panel. The numbers come directly from the script `benchmark_all_estimators.py` bundled with the library. The script can be rerun on any machine to verify the table.

The structural estimators recover the ground truth to four decimal places. The neural and sieve approximation estimators recover the same ground truth at higher computational cost. The maximum causal entropy estimator returns the same parameter vector as the nested fixed point estimator, confirming the theoretical equivalence under linear features. The inverse soft Q-learning estimator recovers the parameters approximately but not exactly, reflecting the chi-squared objective rather than the maximum likelihood objective. The adversarial estimator recovers the parameters at the cost of an order of magnitude longer runtime. The behavioral cloning baseline does not estimate structural parameters but its log-likelihood is reported for comparison. All twelve estimators share the same standard error and identification pipeline, which means that the user can switch between any of them by changing one line of code.

Estimator	θ_1	RC	log-lik.	time (s)	cos. sim.
NFXP	0.001231	3.0115	−4263.20	0.1	1.0000
CCP	0.001230	3.0113	−4263.20	0.1	1.0000
MPEC	0.001231	3.0115	−4263.20	0.5	1.0000
SEES	0.009546	2.9295	−4268.31	1.5	1.0000
NNES	0.031470	3.0727	−4264.00	16.0	1.0000
TD-CCP	0.001084	2.9430	−4264.97	129.6	1.0000
MCE-IRL	0.001231	3.0115	−4263.20	0.1	1.0000
GLADIUS	0.001480	2.3675	−4264.45	10.4	1.0000
AIRL	0.010326	3.8970	−4467.10	607.7	1.0000
IQ-Learn	−0.066029	1.7809	−4678.13	0.0	0.9993
f-IRL	-0.000000	0.0000	−13862.95	133.8	0.0609
BC	—	—	−4238.07	0.0	—

Table 3. Cross-estimator benchmark on the Rust bus panel with ninety mileage states, two actions, and a discount factor of 0.9999. Columns report the operating cost coefficient θ_1, the replacement cost RC, the maximized log-likelihood, the wall-clock time on a single central processing unit thread, and the cosine similarity between the fitted parameter vector and the ground truth. Behavioral cloning does not estimate structural parameters. Reproduced by `examples/rust-bus-engine/benchmark_all_estimators.py` with seed 42 on Apple M2 Pro CPU, **econirl** 0.0.4, **JAX** 0.4.30.

7 Computational details

The illustrations in Section 6 were produced with **econirl** version 0.0.4 on Python 3.11. The package supports Python 3.10 or newer. The required dependencies are **JAX** 0.4.30 or newer, **Equinox** 0.11 or newer, **Optax** 0.2 or newer, **Optimistix** 0.0.7 or newer, **Lineax** 0.0.5 or newer, **NumPy** 1.24 or newer, **Pandas** 2.0 or newer, **SciPy** 1.10 or newer, and **Matplotlib** 3.7 or newer. The library is published to the Python Package Index and installs through the standard command

```
pip install econirl.
```

The development version with optional testing dependencies installs through `pip install "econirl[dev]"`. The package documentation is hosted at https://econirl.readthedocs.io/en/latest/index.html.

The wall-clock numbers reported in Section 6 were measured on a single machine running macOS 14.5 on an Apple M2 Pro central processing unit with 32 gigabytes of memory. The graphics processing unit measurement for the Keane-Wolpin illustration used a single NVIDIA A100 with 40 gigabytes of memory through Google Colab. **JAX** dispatches automatically to the available accelerator, so the user code did not change between the central processing unit and the graphics processing unit run.

All scripts in Appendix A use a fixed random seed of 42 to ensure that the numbers in this paper can be reproduced exactly. Each script writes its output to a file named after the figure or table that consumes it. The full paper rebuild requires running the eleven scripts in the appendix and then recompiling the LaTeX source.

8 Summary and roadmap

econirl unifies twelve estimators for dynamic discrete choice and inverse reinforcement learning under a single result-object contract with a shared post-estimation pipeline. The shared pipeline

provides four standard error methods, identification diagnostics, profile likelihood, hypothesis testing, counterfactual simulation, and visualization. Every improvement to the inference layer benefits every estimator at once. Every new estimator added to the library inherits the inference layer for free. The user picks the estimator that fits the problem and uses the same workflow regardless.

The three illustrations in Section 6 demonstrate the value of the unified workflow. The same Rust bus engine replacement panel admits both a structural likelihood fit through the nested fixed point estimator and an inverse reinforcement learning fit through the maximum causal entropy estimator. The two estimators recover the same parameter vector to four decimal places, confirming the theoretical equivalence under linear features in a setting where running both estimators required only a one-line code change. The Keane-Wolpin occupational choice panel demonstrates that the neural value approximation family scales to state spaces too large for tabular methods while preserving the unified inference workflow. The cross-estimator benchmark in Table 3 demonstrates that all twelve estimators interoperate on the same data with the same outputs available for downstream analysis.

Three extensions are on the roadmap. Continuous-action problems will be supported through a Gauss-Hermite quadrature integration of the action space, which fits the existing protocol with minor changes to the choice probability computation. Dynamic games will be supported through a multi-agent extension that adds a best-response iteration to the existing inner loop. Bayesian inference will be supported through a Hamiltonian Monte Carlo sampler over the parameters, taking advantage of the JAX-computed gradients that the package already produces for maximum likelihood estimation. Each of these extensions preserves the result-object contract that defines the library.

econirl is open source under the MIT license. Documentation, installation instructions, and worked examples are at `https://econirl.readthedocs.io/en/latest/index.html`.

Acknowledgments

We thank seminar participants at Georgetown University for valuable feedback on early versions of the package. We thank the maintainers of **JAX**, **imitation**, and **pyblp** for the open-source infrastructure that this work builds on. Any remaining errors are ours.

References

Karun Adusumilli and Dita Eckardt. Temporal-difference estimation of dynamic discrete choice models. Working paper, 2025.

Victor Aguirregabiria and Pedro Mira. Swapping the nested fixed point algorithm: A class of estimators for discrete Markov decision models. *Econometrica*, 70(4):1519–1543, 2002.

James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs, 2018. http://github.com/jax-ml/jax.

Haoyang Cao, Samuel N. Cohen, and Lukasz Szpruch. Identifiability in inverse reinforcement learning, 2021. Advances in Neural Information Processing Systems.

Christopher Conlon and Jeff Gortmaker. Best practices for differentiated products demand estimation with PyBLP. *RAND Journal of Economics*, 51(4):1108–1161, 2020.

Justin Fu, Katie Luo, and Sergey Levine. Learning robust rewards with adversarial inverse reinforcement learning. In *International Conference on Learning Representations*, 2018.

Janos Gabler and Tobias Raabe. respy: A Python package for the simulation and estimation of Eckstein-Keane-Wolpin models, 2020. https://respy.readthedocs.io.

Divyansh Garg, Shuvam Chakraborty, Chris Cundy, Jiaming Song, and Stefano Ermon. IQ-learn: Inverse soft-Q learning for imitation. In *Advances in Neural Information Processing Systems*, 2021.

Adam Gleave, Mohammad Taufeeque, Juan Rocamonde, Erik Jenner, Steven H. Wang, Sam Toyer, Maximilian Ernestus, Nora Belrose, Scott Emmons, and Stuart Russell. imitation: Clean Imitation Learning Implementations, 2022. arXiv:2211.11972.

V.J̃oseph Hotz and Robert A. Miller. Conditional choice probabilities and the estimation of dynamic models. *Review of Economic Studies*, 60(3):497–529, 1993.

Fedor Iskhakov, Jinhyuk Lee, John Rust, Bertel Schjerning, and Kyoungwon Seo. Comment on "constrained optimization approaches to estimation of structural models". *Econometrica*, 84(1): 365–370, 2016.

Enoch H. Kang, Hema Yoganarasimhan, and Lalit Jain. An empirical risk minimization approach for offline inverse RL and dynamic discrete choice model. *arXiv preprint arXiv:2502.14131*, 2025.

Michael P. Keane and Kenneth I. Wolpin. The career decisions of young men. *Journal of Political Economy*, 105(3):473–522, 1997.

Kuno Kim, Shivam Garg, Kirankumar Shiragur, and Stefano Ermon. Reward identification in inverse reinforcement learning, 2021. International Conference on Machine Learning.

Yong-Jun Lee, K. Sudhir, and Yifan Wang. Adversarial inverse reinforcement learning with unobserved heterogeneity, 2026. Working paper.

Yao Luo and Peijun Sang. Efficient estimation of structural models via sieves. Working paper, University of Toronto, 2024.

Hoang Nguyen. Neural networks for efficient estimation of high-dimensional dynamic discrete choice models. Working paper, Georgetown University, 2025.

Tianwei Ni, Harshit Sikchi, Yufei Wang, Tejus Gupta, Lisa Lee, and Benjamin Eysenbach. f-IRL: Inverse reinforcement learning via state marginal matching, 2021. Conference on Robot Learning.

John Rust. Optimal replacement of GMC bus engines: An empirical model of Harold Zurcher. *Econometrica*, 55(5):999–1033, 1987.

Che-Lin Su and Kenneth L. Judd. Constrained optimization approaches to estimation of structural models. *Econometrica*, 80(5):2213–2230, 2012.

Brian D. Ziebart. *Modeling Purposeful Adaptive Behavior with the Principle of Maximum Causal Entropy*. PhD thesis, Carnegie Mellon University, 2010.

Brian D. Ziebart, Andrew Maas, J. Andrew Bagnell, and Anind K. Dey. Maximum entropy inverse reinforcement learning. In *Proceedings of the 23rd AAAI Conference on Artificial Intelligence*, pages 1433–1438, 2008.

A Reproducing every figure and table

Every numbered figure and table in this paper is produced by exactly one script. To regenerate the paper end to end, run each script below from the repository root. All scripts use a fixed random seed of 42 and write their output to the location named in the third column.

Element	Script	Output
Listing 1	`code_snippets/teaser_nfxp_rust.py`	console
Table 1	`code_snippets/table1_estimator_taxonomy.py`	`figures/table1.tex`
Table 2	`code_snippets/table2_library_comparison.py`	`figures/table2.tex`
Listing 2	`code_snippets/listing2_nfxp_example.py`	console
Figure 1	`code_snippets/fig1_rust_bus_ccp.py`	`figures/fig1.pdf`
Figure 2	`code_snippets/fig2_rust_bus_value.py`	`figures/fig2.pdf`
Listing 3	`code_snippets/listing3_mce_irl_example.py`	console
Figure 3	`code_snippets/fig3_mce_irl_reward.py`	`figures/fig3.pdf`
Listing 4	`code_snippets/listing4_nnes_example.py`	console
Figure 4	`code_snippets/fig4_keane_wolpin_policy.py`	`figures/fig4.pdf`
Table 3	`examples/rust-bus-engine/benchmark_all_estimators.py`	`benchmark_results.csv`

Table 4. Mapping from each numbered element of the paper to a single reproducing script.

The combined runtime of all eleven scripts is approximately one hour on the central processing unit configuration documented in Section 7, dominated by the adversarial inverse reinforcement learning estimator in Table 3. The Keane-Wolpin script accelerates by roughly a factor of five on a graphics processing unit.